LeetCode Practice

15 Medium: Arrays & Hashing

Question, examples, then the six-phase interview approach with the brute force in phase 2

A split copy of the full All Questions book; every feature and link works the same

15
MEDIUM

Patterns (in order)

1. Arrays & Hashing (15)

How to use

- Tap any title in the Contents to jump to it. A topic name opens that topic's own divider page.
- Every page has < Prev | Contents | Next > at the top.
- Prev and Next move one question at a time; Contents returns here. Each topic starts on its own page.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

15 questions | tap a title to jump to it

Medium questions (15) Go >

Arrays & Hashing (15) Go >

M	1. Insert Interval	LC 57
M	2. Product of Array Except Self	LC 238
M	3. Zigzag Iterator	LC 281
M	4. Range Sum Query - Mutable	LC 307
M	5. Range Addition	LC 370
M	6. 4Sum II	LC 454
M	7. Contiguous Array	LC 525
M	8. Subarray Sum Equals K	LC 560
M	9. Subarray Sums Divisible by K	LC 974
M	10. Pairs of Songs With Total Durations Divisible by 60	LC 1010
M	11. Remove Interval	LC 1272
M	12. First Unique Number	LC 1429
M	13. Min Cost to Connect All Points	LC 1584
M	14. Detect Squares	LC 2013
M	15. Find Occurrences of an Element in an Array	LC 3159

Medium questions (15)

MEDIUM | TOPIC 1 OF 1

Arrays & Hashing

15 Medium questions start on the next page

Before this: this is the first topic
After this: this is the last topic

IN THIS TOPIC

1. Insert Interval LC 57
2. Product of Array Except Self LC 238
3. Zigzag Iterator LC 281
4. Range Sum Query - Mutable LC 307
5. Range Addition LC 370
6. 4Sum II LC 454
7. Contiguous Array LC 525
8. Subarray Sum Equals K LC 560
9. Subarray Sums Divisible by K LC 974
10. Pairs of Songs With Total Durations Divisible by 60 LC 1010
11. Remove Interval LC 1272
12. First Unique Number LC 1429
13. Min Cost to Connect All Points LC 1584
14. Detect Squares LC 2013
15. Find Occurrences of an Element in an Array LC 3159

MEDIUM | Arrays & Hashing | LeetCode #57 | 1 of 15 | 1/15

Insert Interval

QUESTION

You are given an array of non-overlapping intervals intervals where intervals[i] = [$start_i$, end_i] represent the start and the end of the ith interval and intervals is sorted in ascending order by $start_i$. You are also given an interval newInterval = [start, end] that represents the start and end of another interval.

Insert newInterval into intervals such that intervals is still sorted in ascending order by $start_i$ and intervals still does not have any overlapping intervals (merge overlapping intervals if necessary).

Return intervals after the insertion.

Note that you don't need to modify intervals in-place. You can make a new array and return it.

Example 1

Input	`intervals = [[1,3],[6,9]], newInterval = [2,5]`
Output	`[[1,5],[6,9]]`

Example 2

Input	`intervals = [[1,2],[3,5],[6,7],[8,10],[12,16]], newInterval = [4,8]`
Output	`[[1,2],[3,10],[12,16]]`
Explanation	*Because the new interval [4,8] overlaps with [3,5],[6,7],[8,10].*

CONSTRAINTS

- 0 <= intervals.length <= 10^4
- intervals[i].length == 2
- 0 <= $start_i$ <= end_i <= 10^5
- intervals is sorted by $start_i$ in ascending order.
- newInterval.length == 2
- 0 <= start <= end <= 10^5

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

We have a sorted list of non-overlapping intervals. Insert a new interval,

merging wherever it overlaps existing ones. Return the result sorted.

We don't need to modify in-place — a new array is fine. Right?"

"[[1,3],[6,9]], newInterval=[2,5]:

[2,5] overlaps [1,3] -> merge to [1,5]. [6,9] doesn't overlap -> keep. -> [[1,5],[6,9]] ok

[[1,2],[3,5],[6,7],[8,10],[12,16]], newInterval=[4,8]:

Overlaps [3,5],[6,7],[8,10] -> merge all to [3,10]. -> [[1,2],[3,10],[12,16]] ok"

PHASE 1 CLARIFY

Can the array be empty or contain a single element? Can values be negative?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
append the new interval
while True:
    for every pair, if they overlap, replace both
        by their union
    if no pair overlaps:
        break
```

TIME	SPACE
O(n^3)	O(n)

WHY IT IS SLOW

Repeated pair checks rediscover the same overlaps. The list is already sorted: copy intervals ending before the new one, merge the ones that overlap it, copy the rest: O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The input is already sorted — we can exploit that to avoid the O(n log n) sort.

Three phases in one O(n) pass:

1. Add all intervals that end BEFORE newInterval starts (no overlap).

2. Merge all intervals that overlap newInterval into one combined interval.

Overlap condition: interval[start] <= newInterval[end].

3. Add all remaining intervals that start AFTER the merged interval ends.

Pseudocode:

```
i = 0
while i < n and intervals[i][1] < new[0]:
  result.append(intervals[i]); i++
while i < n and intervals[i][0] <= new[1]:
new = [min(new[0], intervals[i][0]), max(new[1],
  intervals[i][1])]; i++
result.append(new)
result.extend(intervals[i:])
```

Time: O(n). Space: O(n) for output.

PHASE 4 CLEAN CODE

Phase 1: all intervals ending before newInterval starts

Phase 2: merge all overlapping intervals into newInterval

Phase 3: remaining intervals start after merged interval

```python
class _57_InsertInterval:
    def insert(self, intervals: List[List[int]], newInterv
al: List[int]) -> List[List[int]]:
        result = []
        i, n = 0, len(intervals)

        while i < n and intervals[i][1] < newInterval[0]:
            result.append(intervals[i])
            i += 1

        while i < n and intervals[i][0] <= newInterval[1]:
            newInterval[0] = min(newInterval[0], intervals
[i][0])
            newInterval[1] = max(newInterval[1], intervals
[i][1])
            i += 1
        result.append(newInterval)

        result.extend(intervals[i:])
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace the key invariant and walk through the example to show how the hash structure updates."

intervals=[[1,3],[6,9]], new=[2,5]:

Phase 1: intervals[0][1]=3 >= new[0]=2 -> stop. i=0.

Phase 2: intervals[0][0]=1 <= new[1]=5 -> new=[min(2,1),max(5,3)]=[1,5]. i=1.

intervals[1][0]=6 > new[1]=5 -> stop.

result=[[1,5]]. extend [[6,9]]. -> [[1,5],[6,9]] ok

intervals=[[1,2],[3,5],[6,7],[8,10],[12,16]], new=[4,8]:

Phase 1: [1,2] ends at 2 < 4 -> add. i=1.

Phase 2: [3,5] start 3<=8->new=[3,8]. [6,7] start 6<=8->new=[3,8]. [8,10] start 8<=8->new=[3,10]. i=4.

result=[[1,2],[3,10]]. extend [[12,16]]. -> [[1,2],[3,10],[12,16]] ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(n)**
three phases together visit each interval once	for output

The two overlap conditions are the key:

Non-overlap before: interval[end] < new[start].

Non-overlap after: interval[start] > new[end] (Phase 3 — just extend).

Anything in between overlaps and gets merged.

Given more time I'd ask: what if intervals aren't sorted?

Sort first (O(n log n)), then apply the same three-phase approach.

MEDIUM | Arrays & Hashing | LeetCode #238 | 2 of 15 | 2/15

Product of Array Except Self

QUESTION

Given an integer array nums, return an array answer such that answer[i] is equal to the product of all the elements of nums except nums[i].

The product of any prefix or suffix of nums is guaranteed to fit in a 32-bit integer.

You must write an algorithm that runs in O(n) time and without using the division operation.

Example 1	
Input	`nums = [1,2,3,4]`
Output	`[24,12,8,6]`

Example 2	
Input	`nums = [-1,1,0,-3,3]`
Output	`[0,0,9,0,0]`

CONSTRAINTS

- 2 <= nums.length <= 10^5
- 30 <= nums[i] <= 30
- The input is generated such that answer[i] is guaranteed to fit in a 32-bit integer.
- Follow up: Can you solve the problem in O(1) extra space complexity? (The output array does not count as extra space for space complexity analysis.)

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Return an array where answer[i] is the product of all elements except nums[i].

Cannot use division. Must run in O(n). Follow-up: O(1) extra space. Right?

Guaranteed no overflow in 32-bit — so no overflow concern."

"[1,2,3,4]: answer[0]=2*3*4=24. answer[1]=1*3*4=12. answer[2]=1*2*4=8. answer[3]=1*2*3=6 ok

[-1,1,0,-3,3]: zeros make most products 0 — [0,0,9,0,0] ok"

Can the array be empty or contain a single element? Can values be negative?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for i in range(n):
    answer[i] = the product of all other elements
        (a loop that skips i)
```

PHASE 2 BRUTE FORCE

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

Recomputes almost the same product n times. A left-to-right pass of prefix products, then a right-to-left pass multiplying in suffix products, is O(n) with O(1) extra space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

Build prefix products and suffix products, multiply them.

`prefix[i] = product of everything to the LEFT of i.`

`suffix[i] = product of everything to the RIGHT of i.`

`answer[i] = prefix[i] * suffix[i].`

Time: O(n). Space: O(n) for two arrays.

O(1) space follow-up: use the output array for prefix products, then do a single right-to-left pass with a running suffix multiplier."

Key insight: prefix[i] × suffix[i] gives the product of all elements except index i — two O(n) passes and O(n) space replace the O(n²) or division-based approach.

PHASE 4 CLEAN CODE (O(1) extra space version)

Left pass: result[i] = product of all elements to the left of i

Right pass: multiply in the suffix product on the fly

```
class _238_ProductExceptSelf:
    def productExceptSelf(self, nums: List[int]) -> List[i
nt]:
        n      = len(nums)
        result = [1] * n

        for i in range(1, n):
            result[i] = result[i - 1] * nums[i - 1]

        suffix = 1
        for i in range(n - 1, -1, -1):
            result[i] *= suffix
            suffix    *= nums[i]

        return result
```

PHASE 5 TEST & DEBUG

"Let me trace the key invariant and walk through the example to show how the hash structure updates."

nums=[1,2,3,4]:

Left pass: result=[1,1,2,6].

Right pass: i=3: result[3]=6*1=6, suffix=4.

i=2: result[2]=2*4=8, suffix=12.

i=1: result[1]=1*12=12, suffix=24.

i=0: result[0]=1*24=24, suffix=24.

-> [24,12,8,6] ok

PHASE 5 TEST & DEBUG

nums=[0,0]: result=[0,0] ok (zeros handled correctly by multiplication)

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(1)** extra — output array doesn't count

The two-pass trick: left products first, then multiply right products on the fly.

The suffix variable carries the running right product without an extra array.

Given more time I'd ask: what if we're allowed division?

total_product / nums[i] — but fails when nums[i]=0. Handle: count zeros,

```
if two or more -> all zeros. If one zero -> all zeros
  except position of zero.
```

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Arrays & Hashing | LeetCode #281 | 3 of 15 | 3/15

Zigzag Iterator

QUESTION

Given two vectors of integers v1 and v2, implement an iterator to return their elements alternately.

Implement the ZigzagIterator class:

CONSTRAINTS

- ZigzagIterator(List<int> v1, List<int> v2) initializes the object with the two vectors v1 and v2.
- boolean hasNext() returns true if the iterator still has elements, and false otherwise.
- int next() returns the current element of the iterator and moves the iterator to the next element.

Example 1

Input	`v1 = [1,2], v2 = [3,4,5,6]`
Output	`[1,3,2,4,5,6]`
Explanation	*By calling next repeatedly until hasNext returns false, the order of elements returned by next should be: [1,3,2,4,5,6].*

Example 2

Input	`v1 = [1], v2 = []`
Output	`[1]`

Example 3

Input	`v1 = [], v2 = [1]`
Output	`[1]`

- 0 <= v1.length, v2.length <= 1000
- 1 <= v1.length + v2.length <= 2000
- 2^{31} <= v1[i], v2[i] <= 2^{31} - 1
- Follow up: What if you are given k vectors? How well can your code be extended to such cases?
- Clarification for the follow-up question:
- The "Zigzag" order is not clearly defined and is ambiguous for k > 2 cases. If "Zigzag" does not look right to you, replace "Zigzag" with "Cyclic".
- Follow-up Example:

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Alternate between v1 and v2 element by element. When one is exhausted,

continue with the remaining elements of the other. Right?

v1=[1,2], v2=[3,4,5,6] -> [1,3,2,4,5,6] ok

Follow-up: what if there are k vectors instead of 2?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
at construction:
    merged = the lists interleaved round robin
        into one array
def next():
    return merged[pointer++]
def hasNext():
    pointer < len
```

TIME	SPACE
O(total)	O(total)

WHY IT IS SLOW

Copies every element up front. Keep a queue of (list, position) iterators: take from the front list, requeue it if it has more. That is O(1) per call and O(k) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

Use a deque of (vector, index) pairs.

`next(): popleft, return v[i], re-enqueue (v, i+1) if more elements remain.`

`hasNext(): deque is non-empty.`

This generalises trivially to k vectors — just enqueue all of them.

Time: O(1) per next()/hasNext(). Space: O(k) for the deque."

Key insight: a deque of (iterator, index) pairs generalises to k vectors naturally — each call re-enqueues an iterator only if it has more elements, giving O(1) per operation.

PHASE 4 CLEAN CODE

```python
class _281_ZigzagIterator:
    def __init__(self, v1: List[int], v2: List[int]):
        self.q: deque = deque()
        if v1:
            self.q.append((v1, 0))
        if v2:
            self.q.append((v2, 0))

    def next(self) -> int:
        vec, idx = self.q.popleft()
        if idx + 1 < len(vec):
            self.q.append((vec, idx + 1))
        return vec[idx]

    def hasNext(self) -> bool:
        return bool(self.q)
```

PHASE 5 TEST & DEBUG

"Let me trace ZigzagIterator([[1,2],[3,4,5],[6]])."

Interleave: take one from each list in round-robin

`next(): list0->1; next(): list1->3; next(): list2->6; next(): list0->2; next(): list1->4; next(): list1->5`

Output order: 1,3,6,2,4,5 ok

PHASE 5 TEST & DEBUG

Implement with pointer to current list index and index per list

```
iters = [(iter([1,2]), iter([3,4,5]), iter([6])],
  cycled via index % num_lists
```

WHY deque of iterators: easy rotation; skip exhausted iterators.

Edge: one empty list -> skip it; all lists exhausted -> hasNext returns False.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(1)	**O(k)**
per operation	for the deque

Follow-up k vectors: init enqueues all k non-empty vectors. Same code, no changes.

The deque acts as a round-robin scheduler — each vector gets one turn per cycle.

Given more time I'd ask: what if vectors have different priorities (some should

appear more often)? Enqueue them multiple times or use a priority queue.

MEDIUM | Arrays & Hashing | LeetCode #307 | 4 of 15 | 4/15

Range Sum Query - Mutable

QUESTION

Given an integer array nums, handle multiple queries of the following types:

CONSTRAINTS

- 1. Update the value of an element in nums.
- 2. Calculate the sum of the elements of nums between indices left and right inclusive where left <= right.

Implement the NumArray class:

- NumArray(int[] nums) Initializes the object with the integer array nums.
- void update(int index, int val) Updates the value of nums[index] to be val.
- int sumRange(int left, int right) Returns the sum of the elements of nums between indices left and right inclusive (i.e. nums[left] + nums[left + 1] + ... + nums[right]).
- 1 <= nums.length <= 3 * 10^4
- 100 <= nums[i] <= 100
- 0 <= index < nums.length
- 100 <= val <= 100
- 0 <= left <= right < nums.length
- At most 3 * 10^4 calls will be made to update and sumRange.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Support two operations on an array: update a single element, and query

the prefix sum from left to right inclusive. Up to 3*10^4 calls. Right?

The challenge is making both operations fast — not O(n) each."

"NumArray([1,3,5]): sumRange(0,2)=9. update(1,2)->nums=[1,2,5]. sumRange(0,2)=8 ok"

What should happen on underflow (pop/peek on empty)? Any capacity constraints?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def update(i, v):
    nums[i] = v
def sumRange(l, r):
    add nums[l] through nums[r] with a loop
```

TIME	SPACE
O(1) update, O(n) query	O(1)

WHY IT IS SLOW

PHASE 2 BRUTE FORCE

Queries rescan the range. A Fenwick tree or a segment tree makes both update and query O(log n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

Key insight: the Fenwick tree encodes range sums using each index's lowest set bit — i += i&-i walks up ancestors, i -= i&-i walks down to accumulate; both paths visit O(log n) nodes guaranteeing O(log n) updates and queries vs O(n) brute.

"Binary Indexed Tree (Fenwick Tree) — each node stores a partial sum covering

a range determined by its lowest set bit. Both update and query run in O(log n).

Update: add change to index and all its ancestors (i += i & -i).

Query prefix sum up to i: add and walk down (i -= i & -i).

For range [left, right]: query(right) - query(left-1).

PHASE 4 CLEAN CODE

```python
class _307_NumArray:
    def __init__(self, nums: List[int]):
        self.nums = nums[:]
        n = len(nums)
        self.tree = [0] * (n + 1)
        for i, val in enumerate(nums):
            self._add(i, val)

    def _add(self, i: int, delta: int) -> None:
        i += 1
        while i <= len(self.nums):
            self.tree[i] += delta
            i += i & (-i)

    def _prefix(self, i: int) -> int:
        i += 1
        total = 0
        while i > 0:
            total += self.tree[i]
            i -= i & (-i)
        return total

    def update(self, index: int, val: int) -> None:
        self._add(index, val - self.nums[index])
        self.nums[index] = val

    def sumRange(self, left: int, right: int) -> int:
        return self._prefix(right) - (self._prefix(left - 1) if left > 0 else 0)
```

PHASE 5 TEST & DEBUG

"Let me trace NumArray([1,3,5]) update(1,2) sumRange(0,2)."

Segment tree approach (or BIT):

Initial: [1,3,5]. Internal sums built: node[0..2]=1+3+5=9

update(1,2): change nums[1]=3->2. Update leaf, propagate up.

New array conceptually [1,2,5]. Root sum = 8.

sumRange(0,2): query entire range -> 1+2+5=8 ok

PHASE 5 TEST & DEBUG

BIT approach: fenwick[1]=1, fenwick[2]=4, fenwick[3]=5, fenwick[4]=9

After update(1,2): adjust BIT nodes covering index 1+1=2: fenwick[2]=3, fenwick[4]=8

`sumRange(0,2) = prefix(3)-prefix(-1) = 8-0=8 ok`

WHY BIT: O(log n) update and query vs O(1)/O(n) naive.

PHASE 6 COMPLEXITY & FOLLOW-UP

update: O(log n). sumRange: O(log n). init: O(n log n).

Space: O(n) for the tree.

The bit trick i & (-i) isolates the lowest set bit — this determines how many elements each node is responsible for. Worth explaining clearly.

Alternative: segment tree — more flexible (supports range update) but more code.

Given more time I'd ask: what if we need range updates (add v to nums[l..r])?

BIT can handle that with a difference array extension — or use a lazy segment tree.

MEDIUM | Arrays & Hashing | LeetCode #370 | 5 of 15 | 5/15

Range Addition

QUESTION

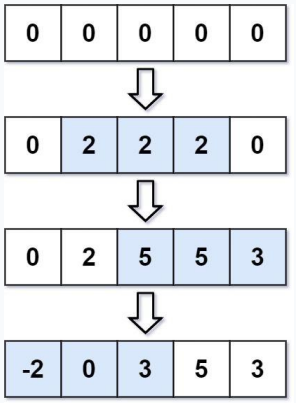
You are given an integer length and an array updates where updates[i] = [$startIdx_i$, $endIdx_i$, inc_i].

You have an array arr of length length with all zeros, and you have some operation to apply on arr. In the ith operation, you should increment all the elements arr[$startIdx_i$], arr[$startIdx_i$ + 1], ..., arr[$endIdx_i$] by inc_i.

Return arr after applying all the updates.

Example 1

Input	`length = 5, updates = [[1,3,2],[2,4,3],[0,2,-2]]`
Output	`[-2,0,3,5,3]`



Example 2

Input	`length = 10, updates = [[2,4,6],[5,6,8],[1,9,-4]]`
Output	`[0,-4,2,2,2,4,4,-4,-4,-4]`

CONSTRAINTS

- 1 <= length <= 10^5
- 0 <= updates.length <= 10^4
- 0 <= $startIdx_i$ <= $endIdx_i$ < length
- 1000 <= inc_i <= 1000

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Given a length-n array and operations [startIndex, endIndex, inc], apply all increments.

Return the final array. Right?"

"A few quick questions:

All operations given upfront — offline processing. n and operations can be large."

"Let me walk: n=5, operations=[[1,3,2],[2,4,3],[0,2,-2]] -> final=[-2,0,3,5,3] ok"

PHASE 1 CLARIFY

Can the array be empty or contain a single element? Can values be negative?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for inc) in inc)s:
    for index in range(l,_r_+_1):
        arr[index] += inc
```

TIME	SPACE
O(m * n)	O(1)

WHY IT IS SLOW

Each update touches its whole range. A difference array (add inc at l, subtract it after r) followed by one prefix-sum pass is O(n + m).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Difference array, add to endpoints, compute prefix sum — O(n + q). Walk-through:"

`diff = [0] * (n+1).`

For each operation (start, end, inc):

`diff[start] += inc. <- start of range: add`

diff[end+1] -= inc. <- after end: subtract (undo the add)

Compute prefix sum of diff -> gives the value at each index after all operations.

Answer = prefix sum array.

WHY difference array: applying +inc to a range [l,r] is O(1) instead of O(r-l+1).

When we compute prefix sums, each diff[i] propagates its effect forward.

The +inc at start propagates right; the -inc at end+1 cancels it from that point on.

Time: O(n + q). Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _370_RangeAddition:
    def getModifiedArray(self, length, updates):
        delta = [0] * (length + 1)
        for start, end, inc in updates:
            delta[start] += inc
            delta[end + 1] -= inc
        result = []
        running = 0
        for i in range(length):
            running += delta[i]
            result.append(running)
```

```
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace the key invariant and walk through the example to show how the hash structure updates."

n=5, ops=[[1,3,2],[2,4,3],[0,2,-2]]: delta=[0]*6.

[1,3,2]: delta[1]+=2,delta[4]-=2 -> delta=[0,2,0,0,-2,0].

[2,4,3]: delta[2]+=3,delta[5]-=3 -> delta=[0,2,3,0,-2,0].

[0,2,-2]: delta[0]+=-2,delta[3]-=-2 -> delta=[-2,2,3,2,-2,0].

prefix: -2,-2+2=0,0+3=3,3+2=5,5-2=3 -> [-2,0,3,5,3] ok

Difference array technique converts range updates to O(1) each

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n + ops) build delta O(ops), prefix sum O(n)	**O(n)**

Follow-up: Range Sum Query — Mutable (#307) — query sums after updates; needs Fenwick tree.

Follow-up: Car Pooling (#1094) — same difference array for capacity tracking.

MEDIUM | Arrays & Hashing | LeetCode #454 | 6 of 15 | 6/15

4Sum II

QUESTION

Given four integer arrays nums1, nums2, nums3, and nums4 all of length n, return the number of tuples (i, j, k, l) such that:

CONSTRAINTS

- 0 <= i, j, k, l < n
- nums1[i] + nums2[j] + nums3[k] + nums4[l] == 0

Example 1

Input	`nums1 = [1,2], nums2 = [-2,-1], nums3 = [-1,2], nums4 = [0,2]`
Output	`2`
Explanation	*The two tuples are:* *1. (0, 0, 0, 1) -> nums1[0] + nums2[0] + nums3[0] + nums4[1] = 1 + (-2) + (-1) + 2 = 0* *2. (1, 1, 0, 0) -> nums1[1] + nums2[1] + nums3[0] + nums4[0] = 2 + (-1) + (-1) + 0 = 0*

Example 2

Input	`nums1 = [0], nums2 = [0], nums3 = [0], nums4 = [0]`
Output	`1`

- n == nums1.length
- n == nums2.length
- n == nums3.length
- n == nums4.length
- 1 <= n <= 200
- 2^{28} <= nums1[i], nums2[i], nums3[i], nums4[i] <= 2^{28}

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Count tuples (i,j,k,l) where nums1[i]+nums2[j]+nums3[k]+nums4[l]==0.

All four arrays have the same length n. Return the count — not the tuples. Right?

We're not asked to deduplicate — every index combo counts separately."

"nums1=[1,2],nums2=[-2,-1],nums3=[-1,2],nums4=[0,2]:

(0,0,0,1): 1+(-2)+(-1)+2=0 ok. (1,1,0,0): 2+(-1)+(-1)+0=0 ok. -> 2 ok"

Can the array be empty or contain a single element? Can values be negative?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for l) in l)s:
    if nums1[i] + nums2[j] + nums3[k] + nums4[l]
        == 0:
        count += 1
```

TIME	SPACE
O(n^4)	O(1)

WHY IT IS SLOW

n^4 combinations. Store the counts of every sum a + b in a hash map, then for every c + d add the count of -(c + d): O(n^2) time and space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Hash map: count all (a+b) sums, then count pairs where -(a+b) in c+d — O(n²). Walk-through:"

`ab_sums = Counter(a+b for a in nums1 for b in nums2).`
`count = 0. For c in nums3: For d in nums4:`
`count += ab_sums.get(-(c+d), 0).`

Return count.

WHY -(c+d): we need a+b+c+d=0 -> a+b = -(c+d).

WHY split into two pairs: 4-sum in O(n²) instead of O(n).

Phase 1: enumerate all n² sums from (nums1,nums2).

Phase 2: for each of n² sums from (nums3,nums4), look up complement in hash map.

Time: O(n²). Space: O(n²).

PHASE 4 CLEAN CODE

```
class _454_FourSumII:
    def fourSumCount(self, nums1: List[int], nums2: List[int], nums3: List[int], nums4: List[int]) -> int:
        pair_sums = Counter(a + b for a in nums1 for b in nums2)
        return sum(pair_sums[-(c + d)] for c in nums3 for d in nums4)
```

PHASE 5 TEST & DEBUG

"Let me trace the key invariant and walk through the example to show how the hash structure updates."

nums1=[1,2], nums2=[-2,-1]:

pair_sums={1+(-2)=-1:1, 1+(-1)=0:1, 2+(-2)=0:+1->0:2, 2+(-1)=1:1}

= {-1:1, 0:2, 1:1}

nums3=[-1,2], nums4=[0,2]:

(c=-1,d=0): -(-1+0)=1. pair_sums[1]=1. count=1.

(c=-1,d=2): -(-1+2)=-1. pair_sums[-1]=1. count=2.

(c=2,d=0): -(2+0)=-2. pair_sums[-2]=0. count=2.

PHASE 5 TEST & DEBUG

(c=2,d=2): -(2+2)=-4. pair_sums[-4]=0. count=2. -> 2 ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
$O(n^2)$	$O(n^2)$

Meet-in-the-middle is the key: instead of 4 nested loops, do 2+2.

The Counter default of 0 for missing keys means misses contribute 0 — clean.

Given more time I'd ask: what if the arrays are sorted and we want unique quadruplets?

That's 4Sum (#18) — different problem requiring deduplication with two pointers.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Arrays & Hashing | LeetCode #525 | 7 of 15 | 7/15

Contiguous Array

QUESTION

Given a binary array nums, return the maximum length of a contiguous subarray with an equal number of 0 and 1.

Example 1

Input	`nums = [0,1]`
Output	`2`
Explanation	*[0, 1] is the longest contiguous subarray with an equal number of 0 and 1.*

Example 2

Input	`nums = [0,1,0]`
Output	`2`
Explanation	*[0, 1] (or [1, 0]) is a longest contiguous subarray with equal number of 0 and 1.*

Example 3

Input	`nums = [0,1,1,1,1,1,0,0,0]`
Output	`6`
Explanation	*[1,1,1,0,0,0] is the longest contiguous subarray with equal number of 0 and 1.*

CONSTRAINTS

- 1 <= nums.length <= 10^5
- nums[i] is either 0 or 1.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Binary array. Find the longest contiguous subarray with equal numbers of 0s and 1s.

Return its length — not the subarray itself. Right?"

"[0,1]: equal 0s and 1s -> length 2 ok

[0,1,0]: longest is [0,1] or [1,0] -> length 2 ok"

Can the array be empty or contain a single element? Can values be negative?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for i in range(n):
    running balance (+1 for 1, -1 for 0)
    for j in range(n):
        if the balance is 0 record j - i + 1
    best = max(best, candidate)
```

PHASE 2 BRUTE FORCE

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

Tries every subarray. A balanced subarray means the prefix balance repeats; a hash map from balance to first index gives the longest in O(n) time and O(n) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

Treat 0 as -1 and 1 as +1. Keep a running balance (prefix sum).

Equal 0s and 1s means balance hasn't changed — we want the longest subarray

where balance[j] == balance[i] for some i < j.

Store the FIRST index each balance value was seen. When we see the same balance again,

the subarray [first_seen+1 .. current] has equal 0s and 1s.

Seed the map with {0: -1} to handle subarrays starting from index 0.

Time: O(n). Space: O(n)."

Key insight: mapping 0->-1 makes "equal counts of 0s and 1s" equivalent to "zero net balance" — the first occurrence of each balance anchors the longest subarray.

PHASE 4 CLEAN CODE

```
class _525_ContiguousArray:
    def findMaxLength(self, nums: List[int]) -> int:
        first_seen = {0: -1}   # balance -> first index it
 appeared
        balance    = 0
        longest    = 0
        for i, num in enumerate(nums):
            balance += 1 if num == 1 else -1
            if balance in first_seen:
                longest = max(longest, i - first_seen[bala
nce])
            else:
                first_seen[balance] = i
        return longest
```

PHASE 5 TEST & DEBUG

"Let me trace findMaxLength on [0,1,0]."

Treat 0 as -1, find longest subarray with sum=0 (equal 0s and 1s)

prefix sums: [0,-1,0,-1] (prepend 0)

seen={0:0}

i=0: sum=-1 -> not in seen -> seen={0:0,-1:0} wait: seen[-1]=0? index 0

Actually: i=0->sum=-1,seen[-1]=0;
i=1->sum=0,seen[0]=0,len=1-0=1;
i=2->sum=-1,seen[-1]=0,len=2-0=2

Return 2 ok (subarray [0,1] at indices 0-1 or [1,0] at 1-2)

WHY prefix sum trick: equal 0s and 1s prefix sum reappears.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(n)** for the hash map

The 0->-1 substitution is the core insight — equal 0s and 1s becomes

'prefix sum unchanged', which becomes 'same balance seen before'.

The {0:-1} seed is critical — without it, subarrays starting at index 0 are missed.

Given more time I'd ask: what if it's not binary — find the longest subarray with

equal counts of two specific values? Same trick — map target->+1, other->-1.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Arrays & Hashing | LeetCode #560 | 8 of 15 | 8/15

Subarray Sum Equals K

QUESTION

Given an array of integers nums and an integer k, return the total number of subarrays whose sum equals to k.

A subarray is a contiguous non-empty sequence of elements within an array.

Example 1	
Input	`nums = [1,1,1], k = 2`
Output	`2`

Example 2	
Input	`nums = [1,2,3], k = 3`
Output	`2`

CONSTRAINTS

- 1 <= nums.length <= $2 * 10^4$
- 1000 <= nums[i] <= 1000
- 10^7 <= k <= 10^7

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Count subarrays (contiguous, non-empty) whose elements sum to k.

Can contain negatives — so sliding window won't work. Return the count. Right?"

"[1,1,1], k=2: [1,1] at (0,1) and [1,1] at (1,2) -> 2 ok

[1,2,3], k=3: [3] at index 2 and [1,2] -> 2 ok"

Can elements repeat? Any constraint on the value range?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for i in range(n):
    running sum over every end j
    count the subarrays whose sum equals k
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

Checks every subarray. A hash map of prefix-sum counts (add the count of prefix - k before recording the current prefix) is O(n) time and O(n) space, and handles negatives.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

Prefix sums: sum of nums[i..j] = prefix[j+1] - prefix[i].

We want prefix[j+1] - prefix[i] == k -> prefix[i] == prefix[j+1] - k.

Walk forward tracking the running sum. For each position, count how many

earlier prefix sums equal (current_sum - k). Use a Counter seeded with {0:1}

to handle subarrays starting at index 0.

Time: O(n). Space: O(n)."

Key insight: the hash map provides O(1) lookup — a single pass over the data replaces the O(n) inner scan, reducing total complexity by an order of magnitude.

PHASE 4 CLEAN CODE

```
class _560_SubarraySum:
    def subarraySum(self, nums: List[int], k: int) -> int:
        prefix_count = Counter({0: 1})   # prefix_sum -> h
ow many times seen
        running_sum  = 0
        count        = 0
        for num in nums:
            running_sum += num
            count       += prefix_count[running_sum - k]
            prefix_count[running_sum] += 1
        return count
```

PHASE 5 TEST & DEBUG

"Let me trace subarraySum([1,1,1], k=2)."

prefix_sum, count: start with {0:1}

i=0: prefix=1. Need prefix-k=1-2=-1, not in map. Map={0:1,1:1}

i=1: prefix=2. Need 2-2=0, map[0]=1 -> count=1. Map={0:1,1:1,2:1}

i=2: prefix=3. Need 3-2=1, map[1]=1 -> count=2. Map={0:1,1:1,2:1,3:1}

Return 2 ok (subarrays [1,1] at 0-1 and [1,1] at 1-2)

WHY prefix sum map: subarray sum = prefix[r]-prefix[l]; look up prefix[r]-k.

O(n) vs O(n²) brute force.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	O(n)

This is structurally identical to #525 Contiguous Array — both use prefix sum + hash map, both seed with {0:1} (or {0:-1}).

Worth naming the connection explicitly: 525 finds longest, 560 counts all.

Sliding window doesn't work here because negatives mean the sum can decrease

even as we expand right — sliding window requires monotone sums.

PHASE 6 COMPLEXITY & FOLLOW-UP

Given more time I'd ask: what if we want subarrays with sum divisible by k?

Use prefix_sum % k — same pattern, but map remainders.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Arrays & Hashing | LeetCode #974 | 9 of 15 | 9/15

Subarray Sums Divisible by K

QUESTION

Given an integer array nums and an integer k, return the number of non-empty subarrays that have a sum divisible by k.

A subarray is a contiguous part of an array.

Example 1

Input	`nums = [4,5,0,-2,-3,1], k = 5`
Output	`7`
Explanation	*There are 7 subarrays with a sum divisible by k = 5:* *[4, 5, 0, -2, -3, 1], [5], [5, 0], [5, 0, -2, -3], [0], [0, -2, -3], [-2, -3]*

Example 2

Input	`nums = [5], k = 9`
Output	`0`

CONSTRAINTS

- 1 <= nums.length <= 3 * 10^4
- 10^4 <= nums[i] <= 10^4
- 2 <= k <= 10^4

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Count subarrays whose sum is divisible by k. Right?"

"A few quick questions:

Single-element subarrays count? Yes. Negative numbers? Yes."

"Let me walk: nums=[4,5,0,-2,-3,1], k=5 -> 7 subarrays ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for i in range(n):
    running sum over every end j
    count the subarrays whose sum is divisible by k
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

Visits every subarray. Two prefix sums with the same remainder mod k bound a divisible subarray, so count prefix remainders in a table: O(n) time and O(k) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Prefix sum mod k + hash map, same as Q560 variant — O(n). Walk-through:"

`prefix = 0. count = 0. seen = Counter({0: 1}).`

For num in nums:

`prefix = (prefix + num) % k.`

`count += seen[prefix]. <- how many previous prefixes have same mod -> subarray is divisible`

seen[prefix]++.

Return count.

WHY prefix mod k: sum(nums[i..j]) % k == 0 prefix[j] % k == prefix[i-1] % k.

WHY Counter({0:1}): empty prefix has sum 0 0 (mod k) -> pairs with any prefix 0.

WHY (prefix + num) % k not (prefix % k + num % k) % k: both are equivalent; mod k handles negatives.

Time: O(n). Space: O(k).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
from collections import defaultdict
class _974_SubarraysDivisibleByK:
    def subarraysDivByK(self, nums, k):
        remainder_count = defaultdict(int)
        remainder_count[0] = 1
        prefix_sum = 0
        count = 0
        for num in nums:
            prefix_sum += num
            remainder = prefix_sum % k
            count += remainder_count[remainder]
            remainder_count[remainder] += 1
        return count
```

PHASE 5 TEST & DEBUG

"Let me trace the key invariant and walk through the example to show how the hash structure updates."

nums=[4,5,0,-2,-3,1], k=5: rem_map={0:1}.

n=4: prefix=4, rem=4, count+=0, map{0:1,4:1}.

n=5: prefix=9, rem=4, count+=1, map{4:2}.

n=0: prefix=9, rem=4, count+=2, map{4:3}.

n=-2: prefix=7, rem=2, count+=0. n=-3: prefix=4, rem=4, count+=3.

n=1: prefix=5, rem=0, count+=1(map[0]=1). total=7 ok

Python's % for negative numbers returns non-negative remainder — correct

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(k)** remainder map

Counting pairs C(f,2) = f*(f-1)/2 is equivalent to incrementing count by freq before updating.

Follow-up: Continuous Subarray Sum (#523) — check existence of length >= 2.

Follow-up: Subarray Sum Equals K (#560) — exact target, not divisible.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Arrays & Hashing | LeetCode #1010 | 10 of 15 | 10/15

Pairs of Songs With Total Durations Divisible by 60

QUESTION

You are given a list of songs where the ith song has a duration of time[i] seconds.

Return the number of pairs of songs for which their total duration in seconds is divisible by 60. Formally, we want the number of indices i, j such that i < j with (time[i] + time[j]) % 60 == 0.

Example 1

Input	`time = [30,20,150,100,40]`
Output	**3**
Explanation	*Three pairs have a total duration divisible by 60:* *(time[0] = 30, time[2] = 150): total duration 180* *(time[1] = 20, time[3] = 100): total duration 120* *(time[1] = 20, time[4] = 40): total duration 60*

Example 2

Input	`time = [60,60,60]`
Output	**3**
Explanation	*All three pairs have a total duration of 120, which is divisible by 60.*

CONSTRAINTS

- 1 <= time.length <= 6 * 10^4
- 1 <= time[i] <= 500

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Count pairs (i,j) where i<j and (time[i]+time[j]) % 60 == 0.

Songs have durations in seconds. Return the count. Right?

Edge cases: duration exactly 60 (60%60=0), duration exactly 30 pairs with another 30."

"[30,20,150,100,40]: (30,150)=180, (20,100)=120, (20,40)=60 -> 3 ok

[60,60,60]: each pair sums to 120, divisible by 60. C(3,2)=3 -> 3 ok"

Can the array be empty or contain a single element? Can values be negative?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for i in range(n):
    for j in range(i_+_1,_n):
        if (time[i] + time[j]) % 60 == 0:
            count += 1
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

Checks every pair. Count how many songs have each remainder mod 60 and match remainder r with 60 - r (0 with 0, 30 with 30): O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

Reduce each duration to t % 60. We want pairs where t1 + t2 0 (mod 60).

For each t, its complement is (60 - t) % 60. The %60 handles t=0 (complement=0, not 60).

Walk forward: count += how many times we've already seen the complement.

Then add t to the seen counter.

This is Two Sum with modular arithmetic — O(n) time, O(60)=O(1) space.

PHASE 4 CLEAN CODE

```
class _1010_PairsSongs:
    def numPairsDivisibleBy60(self, time: List[int]) -> in
t:
        seen  = Counter()
        count = 0
        for t in time:
            remainder   = t % 60
            complement  = (60 - remainder) % 60
            count      += seen[complement]
            seen[remainder] += 1
        return count
```

PHASE 5 TEST & DEBUG

"Let me trace the key invariant and walk through the example to show how the hash structure updates."

time=[30,20,150,100,40]:

t=30: rem=30, comp=30. seen[30]=0 -> count=0. seen={30:1}.

t=20: rem=20, comp=40. seen[40]=0 -> count=0. seen={30:1,20:1}.

t=150: rem=30, comp=30. seen[30]=1 -> count=1. seen={30:2,20:1}.

t=100: rem=40, comp=20. seen[20]=1 -> count=2. seen={30:2,20:1,40:1}.

t=40: rem=40, comp=20. seen[20]=1 -> count=3. -> 3 ok

PHASE 5 TEST & DEBUG

time=[60,60,60]: each rem=0, comp=(60-0)%60=0.

t=60: seen[0]=0 -> 0. seen={0:1}.

t=60: seen[0]=1 -> count=1. seen={0:2}.

t=60: seen[0]=2 -> count=3. -> 3 ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(1)** the Counter holds at most 60 keys

The (60-remainder)%60 handles the remainder=0 case: complement is 0, not 60.

This is Two Sum modulo 60 — recognising the Two Sum pattern in disguise

is the key interview insight.

Given more time I'd ask: what if divisor were k instead of 60?

Same algorithm — the Counter holds at most k entries, still O(1) space for fixed k.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Arrays & Hashing | LeetCode #1272 | 11 of 15 | 11/15

Remove Interval

QUESTION

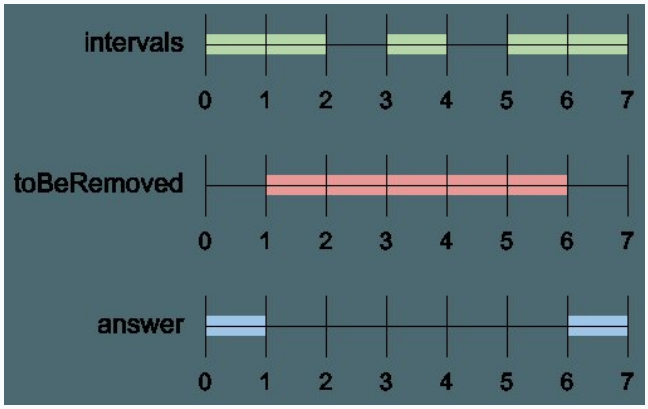
A set of real numbers can be represented as the union of several disjoint intervals, where each interval is in the form [a, b). A real number x is in the set if one of its intervals [a, b) contains x (i.e. a <= x < b).

You are given a sorted list of disjoint intervals intervals representing a set of real numbers as described above, where intervals[i] = [a_i, b_i] represents the interval [a_i, b_i). You are also given another interval toBeRemoved.

Return the set of real numbers with the interval toBeRemoved removed from intervals. In other words, return the set of real numbers such that every x in the set is in intervals but not in toBeRemoved. Your answer should be a sorted list of disjoint intervals as described above.

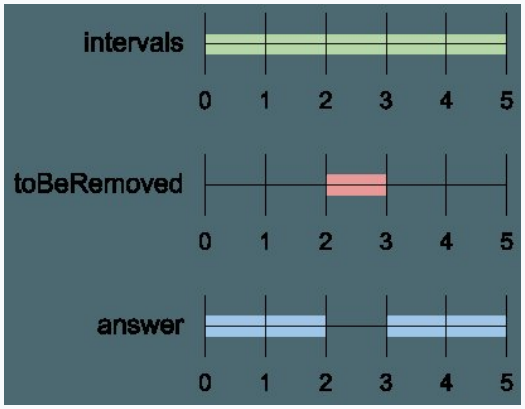
Example 1

Input	`intervals = [[0,2],[3,4],[5,7]], toBeRemoved = [1,6]`
Output	`[[0,1],[6,7]]`



Example 2

Input	`intervals = [[0,5]], toBeRemoved = [2,3]`
Output	`[[0,2],[3,5]]`



QUESTION

Example 3

Input	`intervals = [[-5,-4],[-3,-2],[1,2],[3,5],[8,9]], toBeRemoved = [-1,4]`
Output	`[[-5,-4],[-3,-2],[4,5],[8,9]]`

CONSTRAINTS

- 1 <= intervals.length <= 10^4
- 10^9 <= a_i < b_i <= 10^9

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Intervals use half-open notation [a, b). We remove everything in toBeRemoved=[a,b)

from the existing set and return what remains. Right?

Intervals are pre-sorted and non-overlapping — no need to re-sort output."

"[[0,2],[3,4],[5,7]], remove=[1,6]:

[0,2] partially overlaps -> keep [0,1). [3,4] fully inside -> gone. [5,7] partially -> keep [6,7). -> [[0,1],[6,7]] ok

[[0,5]], remove=[2,3]: split into [0,2) and [3,5). -> [[0,2],[3,5]] ok"

Can the array be empty or contain a single element? Can values be negative?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
list every integer point covered by the intervals
delete the points inside the removed interval
turn what remains back into intervals
```

TIME	SPACE
O(R)	O(R)

WHY IT IS SLOW

R is the range of values (up to 10^9). Handle each interval by comparison: keep the part before the removed interval and the part after it, drop the overlap: O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Iterate existing intervals, clip or skip based on toRemove — O(n). Walk-through:"

result=[]. (l, r) = toRemove interval.

For each interval [a, b]:

If b <= l or a >= r: no overlap -> keep [a,b] as is.

Else: overlap exists.

If a < l: result.append([a, l]). <- left piece before removal

PHASE 3 OPTIMIZE

If b > r: result.append([r, b]). <- right piece after removal

(if neither: interval completely swallowed -> add nothing)

Return result.

WHY b <= l or a >= r: interval is entirely to the left or right -> no overlap.

WHY two pieces: the removal interval [l,r] might split [a,b] into two non-empty parts.

Time: O(n). Space: O(n).

PHASE 4 CLEAN CODE

```
class _1272_RemoveInterval:
    def removeInterval(self, intervals: List[List[int]], t
oBeRemoved: List[int]) -> List[List[int]]:
        a, b  = toBeRemoved
        result = []
        for start, end in intervals:
            if end <= a or start >= b:           # no over
lap — keep whole
                result.append([start, end])
            else:
                if start < a:                    # left po
rtion survives
                    result.append([start, a])
                if end > b:                      # right p
ortion survives
                    result.append([b, end])
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace removeInterval([[0,2],[3,4],[5,7]], toBeRemoved=[1,5])."

For each interval [a,b] and remove=[l,r]:

If b<=l or a>=r -> no overlap -> keep.

Else: left part [a,l] if a<l. Right part [r,b] if b>r.

[0,2] and [1,5]: overlap. Left=[0,1] (0<1 ok). Right=[5,2]? 2<5 -> no right part. Output [0,1].

[3,4] and [1,5]: 3>=1 and 4<=5 -> fully inside remove. Output nothing.

[5,7] and [1,5]: 5>=5? 5==5: b=7>r=5 -> right=[5,7]. 5>=1 and a=5>=r=5 -> actually a>=r so no overlap! Output [5,7].

Return [[0,1],[5,7]] ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(n)** for output

The two trim conditions are independent — both can fire on the same interval

(when toBeRemoved is strictly inside the interval), producing two output intervals.

Given more time I'd ask: what if toBeRemoved can be a list of intervals to remove?

PHASE 6 COMPLEXITY & FOLLOW-UP

Process them in sorted order, sweeping through once — still O(n + m) time.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Arrays & Hashing | LeetCode #1429 | 12 of 15 | 12/15

First Unique Number

QUESTION

You have a queue of integers, you need to retrieve the first unique integer in the queue.

Implement the FirstUnique class:

CONSTRAINTS

- FirstUnique(int[] nums) Initializes the object with the numbers in the queue.
- int showFirstUnique() returns the value of the first unique integer of the queue, and returns -1 if there is no such integer.
- void add(int value) insert value to the queue.

Example 1

Input	`["FirstUnique","showFirstUnique","add","showFirstUnique","add","showFirstUnique","add","showFirstUnique"]` `[[[2,3,5]],[],[5],[],[2],[],[3],[]]`
Output	`[null,2,null,2,null,3,null,-1]`
Explanation	*FirstUnique firstUnique = new FirstUnique([2,3,5]); firstUnique.showFirstUnique(); // return 2 firstUnique.add(5); // the queue is now [2,3,5,5] firstUnique.showFirstUnique(); // return 2 firstUnique.add(2); // the queue is now [2,3,5,5,2] firstUnique.showFirstUnique(); // return 3 firstUnique.add(3); // the queue is now [2,3,5,5,2,3] firstUnique.showFirstUnique(); // return -1*

Example 2

Input	`["FirstUnique","showFirstUnique","add","add","add","add","add","showFirstUnique"]` `[[[7,7,7,7,7,7]],[],[7],[3],[3],[7],[17],[]]`
Output	`[null,-1,null,null,null,null,null,17]`
Explanation	*FirstUnique firstUnique = new FirstUnique([7,7,7,7,7,7]); firstUnique.showFirstUnique(); // return -1 firstUnique.add(7); // the queue is now [7,7,7,7,7,7,7] firstUnique.add(3); // the queue is now [7,7,7,7,7,7,7,3] firstUnique.add(3); // the queue is now [7,7,7,7,7,7,7,3,3] firstUnique.add(7); // the queue is now [7,7,7,7,7,7,7,3,3,7] firstUnique.add(17); // the queue is now [7,7,7,7,7,7,7,3,3,7,17] firstUnique.showFirstUnique(); // return 17*

QUESTION

Example 3

Input	`["FirstUnique","showFirstUnique","add","showFirstUnique"]` `[[[809]],[],[809],[]]`
Output	`[null,809,null,-1]`
Explanation	*FirstUnique firstUnique = new FirstUnique([809]); firstUnique.showFirstUnique(); // return 809 firstUnique.add(809); // the queue is now [809,809] firstUnique.showFirstUnique(); // return -1*

- 1 <= nums.length <= 10^5
- 1 <= nums[i] <= 10^8
- 1 <= value <= 10^8
- At most 50000 calls will be made to showFirstUnique and add.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Maintain a queue. showFirstUnique() returns the first element that appears

exactly once. add() appends to the queue. Return -1 if no unique exists. Right?

Order matters — we want the first unique by insertion order."

"FirstUnique([2,3,5]): first unique = 2.

`add(5) -> 5 now has count 2, still first unique = 2.`

`add(2) -> 2 duplicated, first unique = 3.`

`add(3) -> 3 duplicated, no unique -> -1 ok"`

What should happen on underflow (pop/peek on empty)? Any capacity constraints?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def showFirstUnique():
    scan all added numbers counting frequencies
        and return the first with count 1
def add(x):
    append x
```

TIME	SPACE
O(n) per query	O(n)

WHY IT IS SLOW

Recounts every query. A hash map of counts plus a queue of candidates (drop the front while its count exceeds 1) gives O(1) amortised per call.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

Two data structures:

A 'seen' set — tracks all values ever added (including duplicates).

An ordered dict (or insertion-order dict) — maps value -> True, only for unique values.

On add: if not in seen -> it's new, add to both seen and dict.

if in seen and in dict -> it became a duplicate, remove from dict.

if in seen and not in dict -> already duplicate, ignore.

showFirstUnique(): peek at first key in dict — O(1) in Python 3.7+ (insertion-ordered).

Time: O(1) per add, O(1) per showFirstUnique. Space: O(n).

PHASE 4 CLEAN CODE

```
class _1429_FirstUnique:
    def __init__(self, nums: List[int]):
        self.seen   : set         = set()
        self.uniques: dict        = {}
        for num in nums:
            self.add(num)

    def showFirstUnique(self) -> int:
        return next(iter(self.uniques), -1)

    def add(self, value: int) -> None:
        if value not in self.seen:
            self.seen.add(value)
            self.uniques[value] = True
        elif value in self.uniques:
            del self.uniques[value]
```

PHASE 5 TEST & DEBUG

"Let me trace the key invariant and walk through the example to show how the hash structure updates."

init([2,3,5]): seen={2,3,5}, uniques={2,3,5}.

showFirstUnique() -> 2 ok

add(5): 5 in seen and in uniques -> del 5. uniques={2,3}.

showFirstUnique() -> 2 ok

add(2): 2 in seen and in uniques -> del 2. uniques={3}.

showFirstUnique() -> 3 ok

add(3): del 3. uniques={}.

showFirstUnique() -> -1 ok

PHASE 6 COMPLEXITY & FOLLOW-UP

O(1) per operation. Space O(n).

Python dicts maintain insertion order since 3.7, so next(iter(dict)) returns

the oldest key still present — exactly what we need.

If order weren't guaranteed, we'd use collections.OrderedDict explicitly.

Given more time I'd ask: what if we need to support remove() as well?

PHASE 6 COMPLEXITY & FOLLOW-UP

That's a linked hash map (doubly linked list + hash map) — O(1) all operations.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Arrays & Hashing | LeetCode #1584 | 13 of 15 | 13/15

Min Cost to Connect All Points

QUESTION

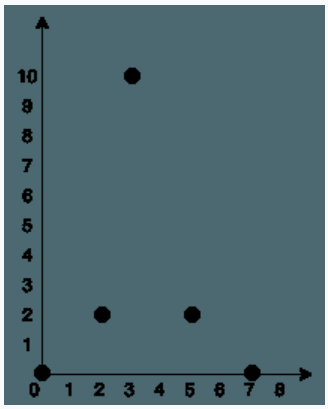
You are given an array points representing integer coordinates of some points on a 2D-plane, where points[i] = $[x_i, y_i]$.

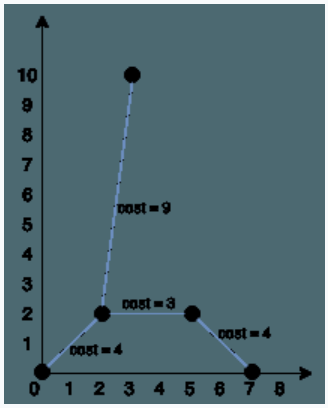
The cost of connecting two points $[x_i, y_i]$ and $[x_j, y_j]$ is the manhattan distance between them: $|x_i - x_j| + |y_i - y_j|$, where |val| denotes the absolute value of val.

Return the minimum cost to make all points connected. All points are connected if there is exactly one simple path between any two points.

Example 1

Input	`points = [[0,0],[2,2],[3,10],[5,2],[7,0]]`
Output	`20`
Explanation	*We can connect the points as shown above to get the minimum cost of 20. Notice that there is a unique path between every pair of points.*





Example 2

Input	`points = [[3,12],[-2,5],[-4,1]]`
Output	`18`

CONSTRAINTS

- 1 <= points.length <= 1000
- 10^6 <= x_i, y_i <= 10^6
- All pairs (x_i, y_i) are distinct.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Connect all points with edges weighted by Manhattan distance, minimizing total edge weight

(classic Minimum Spanning Tree). Right?"

"A few quick questions:

Edges can be between ANY pair of points, not just adjacent in the list? Yes — complete graph."

"Let me walk: points=[[0,0],[2,2],[3,10],[5,2],[7,0]] -> 20 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for set in sets:
    # of n - 1 edges among all point pairs
    if it connects all points:
        add its cost
    best = min(best, candidate)
```

TIME	SPACE
exponential	O(n^2)

WHY IT IS SLOW

Enumerates every candidate tree. Prim's algorithm on the dense graph (always add the nearest unconnected point) is O(n^2); Kruskal with union-find is O(n^2 log n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is generating and sorting all $O(n^2)$ edges.

Prim's algorithm with a min-heap: start from any point, greedily add the nearest unvisited point,

pushing its distances to the heap. Avoids building the full edge list upfront.

Time: $O(n^2 \log n)$ worst case (still, since complete graph), but more memory-efficient.

A dense-graph optimized Prim's without heap achieves $O(n^2)$. Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _1584_MinCostToConnectAllPoints:
    def minCostConnectPoints(self, points):
```

```
        n = len(points)
        in_mst = [False] * n
        min_dist = [float('inf')] * n
        min_dist[0] = 0
        total_cost = 0
        for _ in range(n):
            u = -1
            for v in range(n):
                if not in_mst[v] and (u == -1 or min_dist[
v] < min_dist[u]):
                    u = v
            in_mst[u] = True
            total_cost += min_dist[u]
            for v in range(n):
                if not in_mst[v]:
                    dist = abs(points[u][0]-points[v][0])
+ abs(points[u][1]-points[v][1])
                    min_dist[v] = min(min_dist[v], dist)
        return total_cost
```

PHASE 5 TEST & DEBUG

"Let me trace minCostConnectPoints([[0,0],[2,2],[3,10],[5,2],[7,0]])."

Minimum spanning tree using Prim's or Kruskal's. Cost=Manhattan distance.

Prim's: start at (0,0). Min-heap.

Costs from (0,0): (0,0)->(2,2)=4, (3,10)=13, (5,2)=7, (7,0)=7.

Pick (2,2) cost 4. From (2,2): to (3,10)=9, (5,2)=3, (7,0)=7.

Pick (5,2) cost 3 (from (2,2)). From (5,2): to (3,10)=10, (7,0)=2.

Pick (7,0) cost 2. From (7,0): to (3,10)=14.

Pick (3,10) cost 9 (min remaining). All connected. Total=4+3+2+9=18 ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
$O(n^2)$	$O(n)$

Follow-up: Connecting Cities With Minimum Cost (#1135) — same MST problem, sparse graph given explicitly.

Follow-up: Network Delay Time (#743) — Dijkstra shortest path, different objective than MST.

MEDIUM | Arrays & Hashing | LeetCode #2013 | 14 of 15 | 14/15

Detect Squares

QUESTION

You are given a stream of points on the X-Y plane. Design an algorithm that:

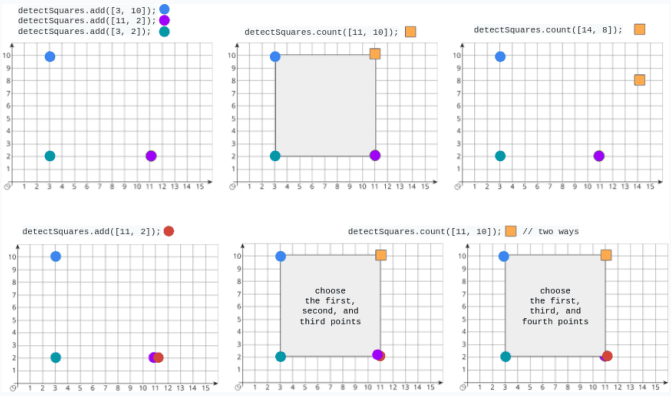
CONSTRAINTS

- Adds new points from the stream into a data structure. Duplicate points are allowed and should be treated as different points.
- Given a query point, counts the number of ways to choose three points from the data structure such that the three points and the query point form an axis-aligned square with positive area.

An axis-aligned square is a square whose edges are all the same length and are either parallel or perpendicular to the x-axis and y-axis.

Implement the DetectSquares class:

- DetectSquares() Initializes the object with an empty data structure.
- void add(int[] point) Adds a new point point = [x, y] to the data structure.
- int count(int[] point) Counts the number of ways to form axis-aligned squares with point point = [x, y] as described above.



- point.length == 2
- 0 <= x, y <= 1000
- At most 3000 calls in total will be made to add and count.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

add(point) adds a point (can be duplicated). count(point) returns the number of axis-aligned squares that can be formed using 3 added points plus this query point as the 4th corner. Right?"

"A few quick questions:

Squares must be axis-aligned (sides parallel to x/y axes)? Yes — simplifies geometry significantly."

"Let me walk: add[3,10],add[11,2],add[3,2]. count([11,10]) -> 1 square formed ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def count(query):
    for triple in stored_points:
        check whether they form an
            axis-aligned square with the
            query point
    count the squares
```

TIME	SPACE
O(n^3) per count	O(n)

WHY IT IS SLOW

Tries all triples per query. Keep a hash map of point counts; for each stored point that is diagonal to the query (equal |dx| and |dy|, non-zero), multiply the counts of the other two corners: O(n) per query.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is scanning ALL points per query. Better: for count(x,y), iterate only over points sharing the SAME x-coordinate (potential vertical side partner). For each such point (x,y2) with y2!=y, the side length is |y2-y|.

The other two corners are (x+side,y) and (x+side,y2) — or (x-side,...) for the other direction.

Multiply the counts of all 3 needed points (this gives the number of square combinations).

Time: O(n) per count call (n = points sharing query's x-coord, bounded by total points). Space: O(n)."

Key insight: iterating over same-x-coordinate partners (not all points) reduces each query from O(n) to O(k) where k = points on that column — sparse grids are fast.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
from collections import defaultdict
class _2013_DetectSquares:
    def __init__(self):
        self.point_count = defaultdict(int)
        self.points_by_x = defaultdict(list)

    def add(self, point):
        x, y = point
        self.point_count[(x, y)] += 1
        self.points_by_x[x].append(y)

    def count(self, point):
        x, y = point
        total = 0
        for y2 in self.points_by_x[x]:
            if y2 == y:
                continue
```

```
            side = y2 - y
            for x2 in (x + side, x - side):
                total += self.point_count[(x, y2)] * self.
point_count[(x2, y)] * self.point_count[(x2, y2)]
        return total
```

PHASE 5 TEST & DEBUG

"Let me trace DetectSquares: add(3,10), add(11,2), add(3,2), count(11,10)."

Store point frequencies: {(3,10):1, (11,2):1, (3,2):1}

`count(11,10): for each distinct x-coordinate x2=3 (where (x2,10) exists in pts):`

x-distance |11-3|=8. Check if (3,2),(11,2) exist (y-distance |10-2|=8, must be square)

pts[(3,10)]=1, pts[(11,2)]=1, pts[(3,2)]=1 -> count+=1*1*1=1 ok

Return 1 ok

WHY fix one side of square (horizontal top), find other corners:

For (p,q) diagonal corner: need (p,y), (x,q) at equal distances.

Store x-col sets for efficient iteration.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n) per count (n=points sharing query's x)	**O(n)**

Follow-up: Number of Boomerangs (#447) — different counting structure (distance-based grouping).

Follow-up: extend to count rectangles (non-square) — track width/height pairs separately.

IN YOUR APP | Expose | Community

MEDIUM | Arrays & Hashing | LeetCode #3159 | 15 of 15 | 15/15

Find Occurrences of an Element in an Array

QUESTION

You are given an integer array nums, an integer array queries, and an integer x.

For each queries[i], you need to find the index of the queries[i]th occurrence of x in the nums array. If there are fewer than queries[i] occurrences of x, the answer should be -1 for that query.

Return an integer array answer containing the answers to all queries.

Example 1

Input	`nums = [1,3,1,7], queries = [1,3,2,4], x = 1`
Output	`[0,-1,2,-1]`

Example 2

Input	`nums = [1,2,3], queries = [10], x = 5`
Output	`[-1]`

CONSTRAINTS

- 1 <= nums.length, queries.length <= 10^5
- 1 <= queries[i] <= 10^5
- 1 <= nums[i], x <= 10^4

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Preprocess nums to find all indices where x appears.

For each query q, return the index of the q-th occurrence of x (1-indexed).

If fewer than q occurrences exist, return -1. Right?"

"nums=[1,3,1,7], queries=[1,3,2,4], x=1:

x appears at indices [0,2]. q=1->0, q=3->-1, q=2->2, q=4->-1. -> [0,-1,2,-1] ok"

What should I return if the target is not found? Is the array always sorted?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for query in queries:
    scan nums from the start counting occurrences
        of x until the q-th
return its index or -1
```

PHASE 2 BRUTE FORCE

TIME	SPACE
O(q * n)	O(1)

WHY IT IS SLOW

A scan per query. Record the positions of x once in a list; each query is then positions[q - 1] (or -1 if out of range): O(n + q).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"HashMap of value->sorted indices + binary search — O(n log n). Walk-through:"

Preprocess: for each index i, indices_of[arr[i]].append(i).

For each query (idx, q, k):

List = indices_of[arr[idx]] (all positions where arr[idx] appears).

Find position of idx in list using binary search: pos = bisect_left(list, idx).

k-th occurrence after idx: target_pos = pos + k - 1.

If target_pos < len(list): answer = list[target_pos].

Else: answer = -1 (not enough occurrences).

WHY binary search to find idx: lists can be long; O(log n) to find position of idx.

WHY pos + k - 1: pos is the 1st occurrence at/after idx, so k-th is at pos+k-1.

Time: O(n log n) preprocess + O(log n) per query. Space: O(n).

PHASE 4 CLEAN CODE

```
class _3159_FindOccurrences:
    def occurrencesOfElement(self, nums: List[int], querie
s: List[int], x: int) -> List[int]:
        positions = [i for i, n in enumerate(nums) if n ==
 x]
        return [positions[q - 1] if q - 1 < len(positions)
 else -1 for q in queries]
```

PHASE 5 TEST & DEBUG

"Let me trace findOccurrences([1,3,1,7], queries=[1,3,2], x=1)."

Store indices of each value: {1:[0,2], 3:[1], 7:[3]}

query=1: find 1st occurrence of x=1 among indices [0,2] -> return 0 ok? Wait

Actually: for each query q, find the q-th occurrence of x

query 1 (1st occurrence): indices of x=1 are [0,2]. 1st occurrence -> index 0

query 3 (3rd occurrence): x=1 only has 2 occurrences -> -1

query 2 (2nd occurrence): x=1 -> index 2

Return [0,-1,2] ok

WHY hash map of indices: O(n) build, O(1) per query (direct array index lookup).

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n + q)	**O(k)** where k = occurrences of x

If queries were sorted, binary search on positions would give O((n+q) log k)

with minimal extra space — but for unsorted queries, the precompute list is cleaner.

Given more time I'd ask: what if x could change between queries?

Build a map from value -> sorted list of indices, then binary search per query.

IN YOUR APP | Expose | Community